Session 7: Let the Data Predict

Business Intelligence & AI

Kerry Back, Rice University

From “What Happened?” to “What Will Happen?”

Atherton Case Debrief

Atherton's Document AI Failures

- **Chunking failures** — long contract clauses split mid-sentence, answers cited incomplete text
- **Access control gaps** — junior analysts saw board-level compensation data
- **Stale documents** — system answered from a superseded compliance policy

The Parallel to Predictive Models

- Stale training data = stale predictions
- Access control still matters — who sees predicted churn scores? Predicted salaries?
- Garbage in, garbage out applies to ML just as much as to document AI

Today’s Learning Objectives

By the end of this session, you will be able to:

- Distinguish classification from regression and identify which applies to a business problem
- Use AI to build and evaluate a gradient boosting model without writing code yourself
- Interpret a confusion matrix, feature importance, and cross-validation scores
- Apply the maker/checker framework to decide whether predictions are trustworthy

Two Types of Prediction

Classification vs. Regression

Classification: “Which category?”

- Will this customer churn? **Yes / No**
- Is this transaction fraudulent? **Fraud / Legitimate**
- Which segment does this lead belong to? **Hot / Warm / Cold**
- Will this employee leave within 12 months?
- Will this loan default?

Regression: “How much?”

- How much will this customer spend next quarter? **$12,400**
- What will demand be in March? **4,200 units**
- What price maximizes margin? **$89.99**
- How many support tickets will we get next week?
- What will Q3 revenue be?

Every business prediction is one of these two types. The type determines how you build the model and how you evaluate it.

Common Business Prediction Problems

Problem	Type	Input Data	Business Value
Customer churn	Classification	Usage patterns, support tickets, tenure	Retain at-risk customers before they leave
Lead scoring	Classification	Demographics, engagement, firmographics	Focus sales effort on highest-probability leads
Fraud detection	Classification	Transaction amount, location, velocity	Block fraudulent transactions in real time
Revenue forecasting	Regression	Historical revenue, pipeline, seasonality	Accurate budgeting and resource allocation
Demand planning	Regression	Sales history, promotions, weather	Optimize inventory and reduce stockouts
Dynamic pricing	Regression	Competitor prices, demand signals, elasticity	Maximize margin without losing volume

Gradient Boosting: The One Method

Why One Method?

70%+

of Kaggle competition winners on tabular data use gradient boosting

XGBoost

most popular implementation — used by every major tech company

#1

first choice for practitioners on real-world business data

Deep learning dominates images and language. For **structured business data** — the kind in your databases and spreadsheets — gradient boosting wins.

How Gradient Boosting Works

Tree 1

Build a small decision tree. It gets some predictions right, some wrong.

→

Find Errors

Look at what Tree 1 got wrong. Focus on those mistakes.

→

Tree 2

Build a new tree that specifically corrects Tree 1's errors.

→

Repeat

Keep adding trees. Each one fixes the remaining errors. Stop when improvement plateaus.

The key insight: No single tree is smart. But hundreds of trees, each correcting the previous one's mistakes, produce remarkably accurate predictions. This is **ensemble learning** — the wisdom of many weak learners.

What Is a Decision Tree?

> A decision tree asks a series of yes/no questions about your data to reach a prediction:

Example: Will this customer churn?

```
Is monthly usage < 5 sessions? ├── YES → Has tenure < 12 months? │ ├── YES → CHURN (85% confidence) │ └── NO
→ Has open support ticket? │ ├── YES → CHURN (70% confidence) │ └── NO → STAY (60% confidence) └── NO → Has
usage declined 3 months in a row? ├── YES → CHURN (65% confidence) └── NO → STAY (90% confidence)
```

Each tree in gradient boosting is **small and simple** — maybe 4-8 questions deep. The power comes from combining hundreds of these simple trees.

The ML Workflow

Five Steps to a Predictive Model

Prepare Data
Clean, explore, handle missing values

→

Split
Train set (80%) and test set (20%)

→

Fit Model
Train gradient boosting on the training set

→

Predict
Apply model to the test set it has never seen

→

Evaluate
Confusion matrix, accuracy, feature importance

Why split the data? If you evaluate a model on the same data it trained on, it is like grading a student on questions they already saw. The test set is the **final exam with new questions**.

Demo: Prompting Claude to Build a Model

Step 1: Explore the Data

PROMPT: I have a CSV file with customer data. Each row is a customer. The columns include tenure, monthly_charges, total_charges, contract_type, payment_method, internet_service, and a column called “churn” that is Yes or No.

Please explore this data: how many rows, any missing values, and what does the distribution of churn look like?

Demo: Claude's Exploration Output

Claude's Response (simulated)

Here's what I found in your dataset:

Dataset overview: - 7,043 customers, 21 columns - No missing values (after converting 11 blank TotalCharges to 0)

Churn distribution: - Did not churn: 5,174 (73.5%) - Churned: 1,869 (26.5%)

Key observations: - Month-to-month contracts churn at 42% vs. 11% for 2-year contracts - Customers with fiber optic internet churn at 42% vs. 25% overall - Average tenure for churned customers: 18 months vs. 38 months for retained customers

⚠️ The dataset is imbalanced (73/27 split). I'll account for this when building the model.

Demo: Building the Model

Step 2: Build and Evaluate

```
PROMPT: Build a gradient boosting classifier to predict churn. Use an 80/20 train-test split and 5-fold
cross-validation.

Show me: 1. The confusion matrix on the test set 2. Precision, recall, and F1 score 3. The top 10 most
important features 4. Cross-validation scores

Explain the results in business terms, not just statistics.
```

Demo: Model Results

Claude's Model Output (simulated)

Model: GradientBoostingClassifier (200 trees, max depth 4)

Confusion Matrix (test set, 1,409 customers): Predicted Stay Predicted Churn Actual Stay 896 139 Actual Churn 102 272

Metrics: - Accuracy: 82.9% - Precision (churn): 66.2% — when we predict churn, we're right 66% of the time - Recall (churn): 72.7% — we catch 73% of actual churners - F1 Score: 69.3%

Cross-validation: 81.4% ± 1.8% (stable across folds)

Evaluating Models

Reading a Confusion Matrix

What the Matrix Tells You

- **True Positives (272):** Correctly predicted churn — these are your wins
- **True Negatives (896):** Correctly predicted stay — no action needed
- **False Positives (139):** Predicted churn, but customer stayed — unnecessary retention spend
- **False Negatives (102):** Predicted stay, but customer churned — **missed opportunity, the costly error**

The Business Question

- What is the **cost** of each error type?
- False positive: you offer a discount to someone who was staying anyway (~$50/customer?)
- False negative: you lose a customer worth $1,200/year in revenue
- **False negatives are 24x more costly** — optimize for recall

Precision vs. Recall: A Business Tradeoff

Precision (66.2%)

- "When we predict churn, how often are we right?"
- 272 correct out of 411 churn predictions
- Higher precision = fewer wasted retention offers
- **Optimize for precision when false positives are expensive**
- Example: fraud alerts that freeze customer accounts

Recall (72.7%)

- "Of all actual churners, how many did we catch?"
- 272 caught out of 374 actual churners
- Higher recall = fewer missed churners
- **Optimize for recall when false negatives are expensive**
- Example: cancer screening, churn prevention, fraud in high-value accounts

You cannot maximize both. Catching more churners (higher recall) means more false alarms (lower precision). The right balance depends on your business costs.

Regression Metrics: R^2 and RMSE

R^2 (R-squared)

- "How much of the variation does the model explain?"
- R^2 = 0.85 means the model explains 85% of variation in the target
- R^2 = 1.0 is a perfect model (suspicious — probably data leakage)
- R^2 = 0.0 means the model is no better than predicting the average
- **Business rule of thumb: $R^2 > 0.7$ is usually useful for planning**

RMSE (Root Mean Squared Error)

- "On average, how far off are the predictions?"
- RMSE = $4,200 means predictions are off by ~$4,200 on average
- In the same units as the target variable (dollars, units, etc.)
- **Ask: "Is this error margin acceptable for the business decision?"**
- RMSE of $4,200 on $50K deals is fine; on $8K deals, it is not

Feature Importance: What Drives Predictions?

Top 10 Features (from our churn model)

```
Feature Importance ──────────────────────────────────────── 1. contract_type 0.231 ← Month-to-month is the #1 risk 2. tenure 0.189 ← New customers churn more 3. monthly_charges 0.142 ← Higher bills → higher churn 4. internet_service_fiber 0.098 ← Fiber customers churn more 5. total_charges 0.074 6. payment_method_electronic 0.062 7. online_security_no 0.048 8. tech_support_no 0.041 9. online_backup_no 0.035 10. paperless_billing 0.028
```

The maker/checker question: Do these make business sense? Contract type and tenure as top predictors? **Yes — that passes the smell test.** If “customer_id” were the top feature, something is very wrong.

The Maker/Checker Framework for Models

1. Better Than Baseline?

- Does the model beat a naive approach?
- Baseline for churn: predict "stay" for everyone → 73.5% accuracy
- Our model: 82.9% accuracy
- **9.4 percentage points better** — real improvement

2. Features Make Sense?

- Do the top predictors match domain knowledge?
- Contract type, tenure, charges → **makes sense**
- Customer ID, row number → **data leakage**
- Ask a domain expert: "Do these surprise you?"

3. Generalizes Well?

- Cross-validation: 81.4% ± 1.8%
- Training accuracy: 85.2%
- Small gap → model is **generalizing**
- If train = 99% but CV = 60% → overfitting

Red Flags: When Not to Trust the Model

🚩 Too Good to Be True

- 99% accuracy usually means data leakage
- The target variable leaked into the features
- Example: including “cancellation_date” to predict churn

🚩 Nonsense Features

- Customer ID, row number, or timestamp as top features
- The model is memorizing, not learning
- Always check feature importance before trusting predictions

🚩 Big Train/Test Gap

- Train accuracy: 98%
- Test accuracy: 65%
- The model memorized training data and fails on new data

🚩 Unstable Cross-Val

- Scores: 85%, 62%, 78%, 91%, 55%
- High variance across folds
- Model is sensitive to which data it sees — not reliable

Hands-On: Build a Predictive Model

Exercise Setup

What You Need

- Open Claude at claude.ai
- Download the customer churn dataset (link in chat)
- Upload the CSV to Claude
- Follow the five guided steps on the next slide

Time: 20 minutes. Work individually. I will circulate to help.

Goal: Not just to build a model, but to **evaluate it** using the framework we just learned. Can you answer the three maker/checker questions about your model?

Five Guided Steps

1

Explore

"Explore this data. How many rows? Missing values? What's the churn rate?"

→

2

Build

"Build a gradient boosting classifier with 80/20 split and cross-validation."

→

3

Evaluate

"Show me the confusion matrix. What's the false negative rate?"

→

4

Interpret

"What are the top 10 features? Do they make business sense?"

→

5

Challenge

"What would a skeptical VP push back on? How would you improve this?"

Bonus prompt: "If I could only act on the top 200 highest-risk customers, which ones would you flag and why?"

Example: Step 5 in Action

Prompt: Challenge the Model

PROMPT: What would a skeptical VP of Customer Success push back on if I presented this model? What are the weaknesses?

Claude's Response (simulated)

A skeptical VP would likely raise these concerns:

1. **"We're missing 27% of churners."** The recall of 72.7% means 102 customers who churned were not flagged. If each lost customer costs $1,200/year, that's $122K in missed revenue.
2. **"Contract type is the top feature — we already know that."** The model confirms what the team already suspects. The VP wants to know: what does the model tell us that we DON'T already know? → Look at features 4–10 for non-obvious insights.
3. **"This is trained on historical data."** If we just changed our pricing or launched a new product, past churn patterns may not predict future behavior. The model needs retraining regularly.
4. **"What do we DO with the predictions?"** A model is only useful if it drives action. What's the intervention for flagged customers? Who owns the follow-up?

Exercise Debrief

Quick Show of Hands

- Who got a model accuracy above 80%?
- Who found that contract type was the top feature?
- Who tried the bonus prompt about flagging specific customers?
- Who found something in the feature importance that surprised them?

The model is the easy part. The hard part is: **what do you do with the predictions?**

From Model to App

Deploy Your Model as a Dashboard

The prompt (typed into Claude Code on the AI+Code Lab)

```
Take the churn prediction model we just built. Create an
interactive web app where a user can enter customer attributes
(contract length, monthly charges, tenure, support tickets)
and get a churn probability with the top 3 risk factors.
```

What the App Produces

- Input form with sliders and dropdowns for customer attributes
- A prediction output: “Churn probability: 73% — High Risk”
- Feature importance bar chart showing the top 3 risk factors for this customer
- Recommended actions: “Offer contract upgrade,” “Assign dedicated support rep,” “Schedule executive check-in”
- A sales manager could use this every day to prioritize retention outreach

The Full Arc

1

Data

Raw customer records from enterprise systems

→

2

Model

Gradient boosting trained on historical patterns

→

3

Dashboard

Web app with inputs, predictions, and actions

→

4

Decision

Sales manager calls the right customer today

From raw data to a decision tool. This is what AI + code gives you: not just answers, but **applications**.

Breakout: Prediction Opportunities

Breakout Instructions

Breakout Exercise (10 minutes, groups of 3-4)

Each person identifies 1-2 prediction problems at their company:

1. **What would you predict?** (Be specific: “customer churn” not “make better predictions”)
2. **Classification or regression?**
3. **What data would you need?** Do you have it? Where does it live?
4. **What is the cost of a false positive vs. false negative?**
5. **What would you DO with the predictions?** Who acts on them?

Report back: Each group shares the most valuable prediction opportunity and the biggest data obstacle.

Report Back

Most Valuable Predictions

- What patterns emerged across groups?
- Which prediction opportunities had the clearest ROI?
- Did anyone identify a prediction that changes a decision process?

Biggest Data Obstacles

- Data does not exist in clean form
- Lives in systems you cannot access
- Would need to combine multiple sources
- **Sound familiar from Sessions 3-4?**

The Action Question

- A prediction without action is a report
- Who receives the predictions?
- What decision changes?
- **This is the capstone question**

Bridge to Capstone

Your Full Toolkit

Sessions 1-2
Personal productivity: charts, docs, analysis

→

Session 3
Enterprise data: structured queries + document Q&A

→

Session 4
Reporting pipelines: data → analysis → deliverable

→

Session 5
Deployment: architecture, security, build vs. buy

→

Session 6
Trust: failure modes, red-teaming, governance

→

Session 7
Prediction: classification, regression, evaluation

Your capstone strategy can draw on **any combination** of these capabilities. Next session: you present.

Capstone Preparation

Your Strategy Document (1-2 pages)

- Problem statement: what business problem does AI solve?
- Proposed solution: which AI capabilities do you use?
- Deployment approach: cloud, on-premise, or hybrid?
- Governance plan: access controls, verification, compliance
- 90-day pilot plan: concrete first steps

Your Presentation (5 minutes)

- Pitch to your executive team (your breakout group)
- Breakout groups vote on strongest strategy
- 5 winners present to full class
- Instructor feedback on: scoped tightly enough? Business case credible? Governance adequate?

Self-check: Use the evaluation rubric provided with the template. Does your Day 1 action actually happen on Day 1?

Homework

Before Session 8 (Monday)

1. **Finalize your strategy document** (1-2 pages using the capstone template)
2. **Prepare your 5-minute presentation** — use the evaluation rubric to self-check
3. **Consider predictive modeling** — if it fits your use case, add it to your strategy with specifics: what would you predict, what data would you need, what action would follow?

Optional: Demos 6-7 show building a RAG system and combining it with a database agent. Relevant if your capstone includes document AI.

Optional: Deploy your churn model as an interactive dashboard on the AI+Code Lab. The prompt is one sentence.

Key Takeaways

Three Things to Remember

AI Builds the Model

- You do not need to write code
- You do not need to understand the math
- You need to write clear prompts and **evaluate the output**

Three Questions to Ask

- Better than baseline?
- Features make business sense?
- Generalizes to new data?
- **If yes to all three, proceed with caution**

Predictions Need Action

- A model without action is just a report
- Who receives predictions?
- What decision changes?
- **This is what separates strategy from spectacle**